

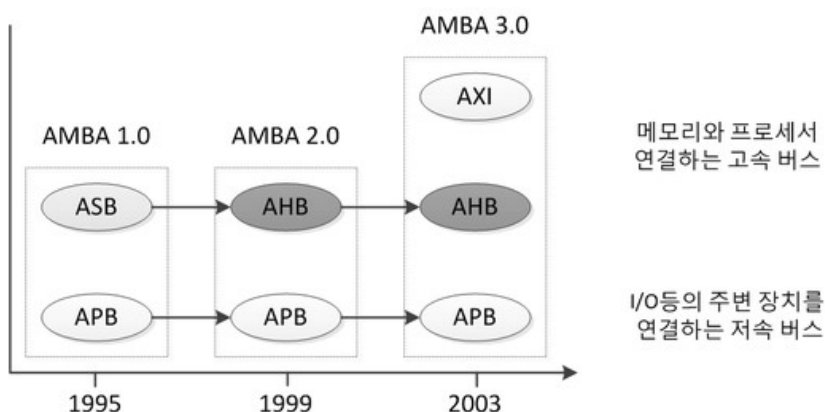
AMBA 란? AMBA / ARM

AMBA (Advanced Microcontroller Bus Architecture)는 영국의 ARM 사가 개발한 시스템 LSI 용 온칩(On-Chip, 칩 내부) 버스 표준이다. (시스템 LSI 나 우리가 흔히 얘기하는 System on Chip(SoC)이나 같은 용어라고 생각 하시면 됩니다.) 컴퓨터를 구입할 때 주로 보게 되는 PCI 와 같은 것들은 장비와 장비 사이에 Data 를 전달하는 Off-chip 버스 라고 생각하시면 됩니다.

온칩 버스를 이용하여 SoC 내부의 각 IP 코어를 연결하고 데이터를 서로 주고받게 됩니다. 이상적으로는, SoC 를 구성하는 모든 업체의 IP 코어를 쉽게 연결할 수 있도록 하는 것이 바람직하지만, 버스의 사양이 통일되지 않으면 상당히 많은 시간이 들어 갑니다. 그래서 ARM 사는 온칩 버스의 사양을 정하고 이를 무상 공개하여 IP 코어 인터페이스의 표준화를 촉진하고 있습니다.

예전에 실험을 위해 OpenCores(<http://opencores.org/>)에 등록된 CAN IP 를 사용한 적이 있었습니다. Xilinx 의 Zynq 나 Altera 의 Excalibur 의 경우 AMBA 를 사용 하고 있지만. 그 당시 CAN IP 는 wishbone bus 로 작성 되어 있어서. 이를 AHB Bus 로 바꾸기 위해 많은 시간을 소모 했던 적이 있습니다. 최근에는 FPGA 를 생산하는 Xilinx 에서 AXI 버스로 IP 의 버스를 통일하여 실험이나 시제품을 만들 때 하나의 버스 시스템을 이해 하면 되므로 많이 편리해졌다는 생각이 듭니다.

AMBA 의 첫 번째 버전인 AMBA 1.0 은 1995 년에 공개되었습니다. 1.0 에서는 ASB (Advanced System Bus)와 APB (Advanced Peripheral Bus)라는 두 버스 사양이 정의되어 있습니다. ASB 는 프로세서와 메모리 및 고성능 장치를 연결하는 버스, APB 는 I/O 등의 주변 장치를 연결하는 버스입니다.



프로세서와 메모리 등을 연결하는 버스 규격을 살펴보면, 1995 년에 발표된 ASB (Advanced System Bus) 규격에서 1999 년에는 고성능 AHB (Advanced High-performance Bus) 규격으로 대체되었으며, 2003 년에는 고성능 시스템 LSI 용 AXI (Advanced eXtensible Interface) 가 추가되었다.

다음 버전 인 AMBA 2.0 은 1999 년에 발표되었습니다. 이 사양은 새롭게 AHB (Advanced High-performance Bus)를 정의하였습니다. AHB 는 SoC 용으로 ASB 보다 진보된 기능을 가지고 있습니다. 구체적으로 설명하면 합성을 쉽게 하기 위하여 상승에지 만의 타이밍을 이용하도록 하고, Tri-State 없이 설계하도록 변경되었습니다.

다음 버전인 AMBA 3.0 은 2003 년 6 월에 발표되었습니다. 이 사양은 AMBA 2.0 AHB 와 APB 외에 AXI (Advanced eXtensible Interface)라는 새로운 버스가 추가되었고. AXI 는 고성능 SoC 의 요구에 부응하기 위해 AHB 사양 보다 높은 성능이 필요한 시스템의 메인 버스를 상정하고 개발 된 인터페이스입니다.

AXI (1) AMBA / ARM

1. AXI 사양

AHB 의 신호와 타이밍은 일반적인 CPU 의 버스 인터페이스와 비슷합니다. 반면 AXI 사양은 성능을 높이기 위해 기존과는 상당히 다른 구조로 되어 있습니다.

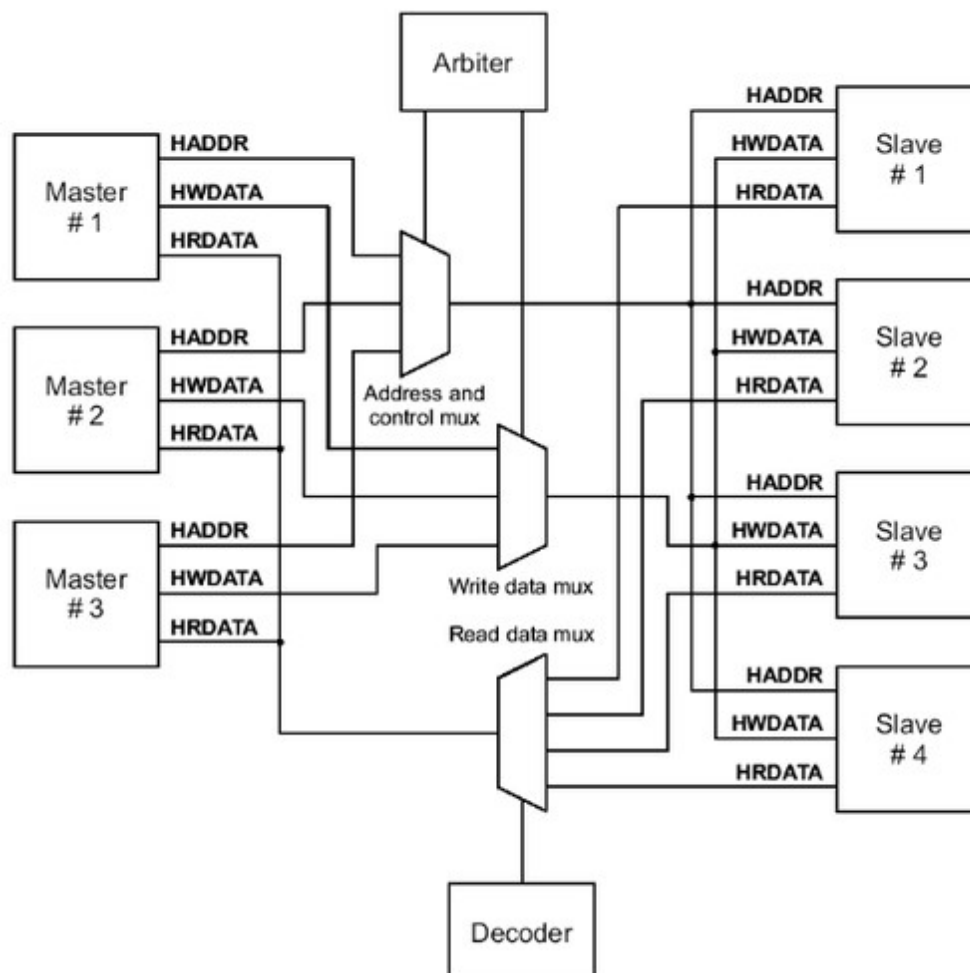
● AXI 에서 도입된 '채널' 이라는 개념

AXI 와 AHB 의 가장 큰 차이는 '채널'이라는 개념의 도입입니다. 채널은 지금까지의 일반적인 버스 구조에는 없었던 개념입니다. 'AXI 가 기존과 어떻게 다른가?' 를 알아보기 위하여 AHB 와 같은 기존의 "버스" 주 1 에 대해서 먼저 알아보도록 하겠습니다.

주 1: 버스라는 개념 자체도 정확한 정의가 있는 것은 아니다. CPU 외부 신호 군 전체를 하나의 버스라고 칭하는 것도 있고, 여러 선으로 구성되어 다양한 정보를 전송하는 것을 버스라고 부르는 경우도 있다.

AHB 는 일반적으로 CPU 등의 버스 (외부 장치와 연결하기 위한 신호)에 사용됩니다. 이 AHB 는 다음과 같은 복수의 버스와 신호로 구성됩니다 (그림 3). 신호 앞에 H 라고 붙는 것은 AHB 의 신호 이기 때문입니다. 또한, APB 버스의 신호는 신호이름 앞에 P 라고 붙게 됩니다.

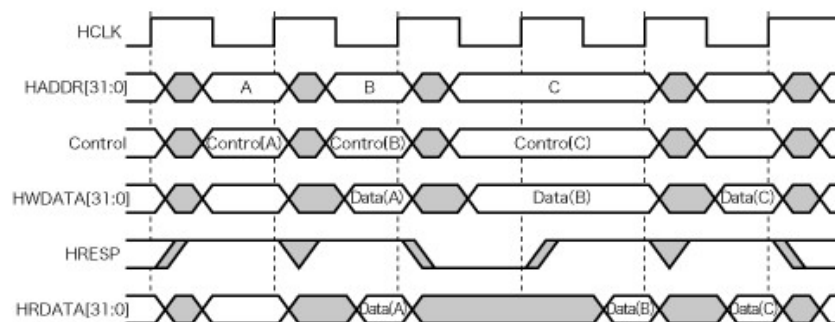
- 주소 버스 (HADDR)
- 읽기 데이터 버스 (HRDATA)
- 쓰기 데이터 버스 (HWDATA)
- 제어 신호
- 응답 신호



[그림 3] AHB 사양의 버스 구조

AHB 버스는 각각의 정보를 나르는 역할만 하게 됩니다(예를 들어 ADDR 은 주소만, WDATA 는 쓰기 자료, RDATA 는 읽기 자료만 이동합니다.). 이러한 버스 사이의 동작 타이밍은 문서에 정의되어 있으며, 이것들을 따라서 설계해야 합니다. 하나의 전송을 위해 각 버스(HADDR 과 HWDATA)가 함께 작동해야 하며, 독립적으로 작동할 수 없습니다.

이러한 버스는 단지 정보를 버스 위에 실어 놓기 만 합니다. 정보를 외부 디바이스에 건네주는 순서는, 다른 버스나 제어 신호와의 타이밍에 의해 결정됩니다. 순서는 그림(4)와 같습니다.



[그림 4] AHB의 타이밍 다이어그램

주소(HADDR)와 제어(Control)신호를 송신해, 그에 대한 데이터를 송신하고, 응답하는 순서로 진행되게 됩니다. 하지만, 속도가 낮은 디바이스가 데이터 준비에 시간이 걸리면 그 동안 주소버스는 다음을 진행하지 않고 대기하게 되는 문제점이 있습니다. 이로 인해 버스 효율이 떨어지게 됩니다.

1) 1 사이클의 동안, 마스터(CPU)가 주소버스(HADDR)에 주소를 싣고, 제어 신호(Control)에는 이에 해당하는 트랜잭션(transaction)^(주 2)의 정보를 싣는다.

2) (데이터 읽기의 경우) 다음 사이클에 슬레이브(디바이스)가 읽기 데이터버스(HRDATA)에 데이터를 싣고, 결과를 응답 신호(HRESP)에 싣는다.

3) (데이터 쓰기의 경우) 마스터가 쓰고자 하는 데이터를 데이터버스(HWDATA)에 싣고, 슬레이브가 결과를 응답 신호(HRESP)에 싣는다.

주 2 ; 명령의 시작부터 명령 처리 종료까지의 한 묶음의 처리를 트랜잭션(transaction)라고 부르고 있다. 예를 들면 읽기 처리의 경우, 읽기 명령을 시작하고 나서 데이터를 읽어낼 때 까지를 가리킨다.

이 방식은 각 버스에 신호가 실리는 차례나 타이밍, 버스 상호의 관계에 의해서 데이터의 전송 처리가 진행됩니다. 따라서, 버스 간의 동작 타이밍은 엄밀하게 정의되어 있고 버스신호만 독립적으로 수행되지는 않습니다.

AHB 버스 문제점중의 하나는, 저속 디바이스와의 데이터 전송이 존재하면 후속의 디바이스와의 데이터 전송을 기다려야 한다는 것입니다. 예를 들면, 주소를 주소 버스에 실린 다음주기는 반드시 그 주소에 대한 데이터를 데이터 버스로 전송해야 합니다. 따라서 저속 장치는 데이터의 준비에 시간이 걸리면 그 동안 주소 버스는 다음 진행을 하지 못하기 때문에 버스의 효율이 떨어지게 됩니다. 또한, 고속 장치가 후속을 전송할 경우 서비스 속도를 살릴 수 없습니다.

반면 AXI 에서는 다음과 같은 신호로 구성되어 있습니다.

- 읽기 주소·채널
- 읽기 데이터·채널
- 쓰기 주소·채널
- 쓰기 데이터·채널
- 쓰기 응답 채널

종래 「xxx 버스」라고 칭하고 있던 것이 「xxx 채널」이 되어 있습니다. AXI 는 각 채널마다 독립적으로 정보를 전송할 수 있습니다. 예를 들어, AHB 에서는 "주소를 전송하면 다음 사이클에서 해당 데이터를 반드시 전달 해야한다" 등의 제약이 있습니다. 하지만, AXI 에서는 채널 간 단계에 대해서 제약이 없습니다. 따라서 다른 채널의 시기에 관계없이 정보를 계속 전송할 수 있습니다. 이처럼 AXI 에서는 버스 사이가 독립적으로 동작하며 정보 스트림을 전송 할 수 있으므로, "채널"이라고 부르고 있습니다.

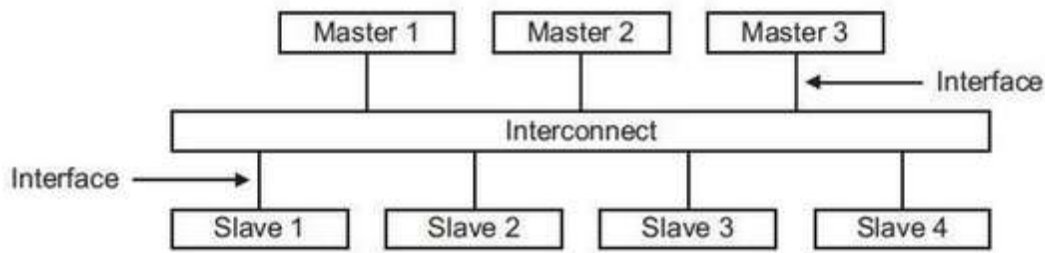
또한, AXI 는 인터페이스 사양이며, 버스에 대한 사양이 아닙니다(그림 5). 따라서, 상호연결 (Interconnect)부분은 문서에서 정의하고 있지 않습니다(주 3). 상호연결 형태로는 다음과 같은 것들을 생각할 수 있습니다.

포인트 투 포인트

싱글 레이어

멀티 레이어

주소는 싱글 레이어, 데이터는 멀티 레이어



[그림 5] AXI 사양 구조

AXI 사양은 인터페이스 사양을 제시한 것이지, 버스 사양에 대한 것은 아닙니다. 따라서 상호연결(Interconnect) 부분은 사양에서 제외 되어있습니다.

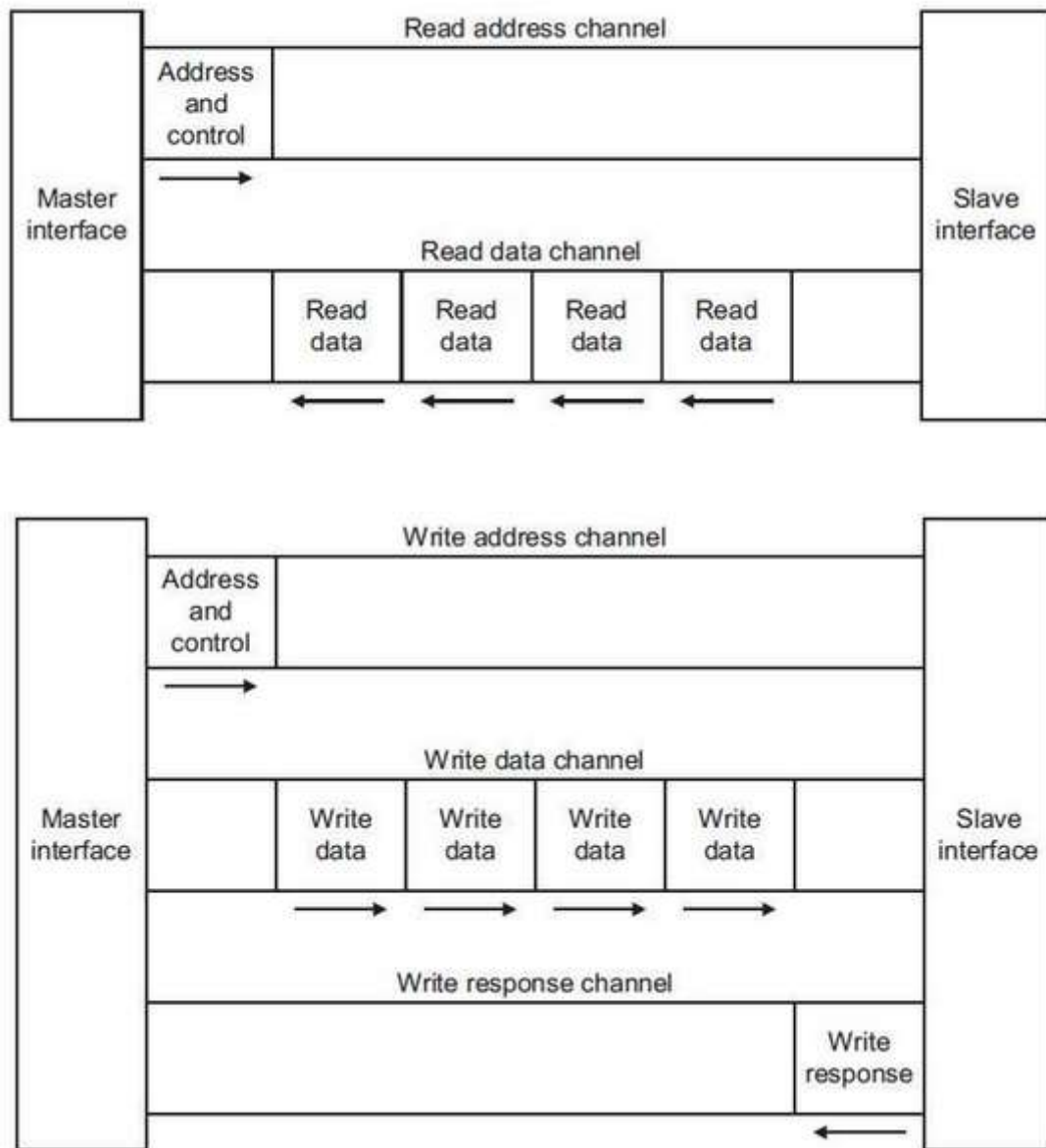
주 3; AHB 의 사양서에는 멀티플렉서로 연결 구성이 실려 있지만, 나중에 고안된 멀티 레이어 AHB 는 상호연결 구현 방식을 사용할 수 있도록 되어져 있습니다.

AXI(2) AMBA / ARM

AXI 의 주요 특징은 다음과 같습니다.

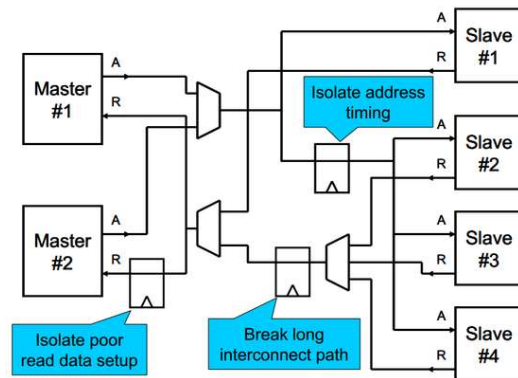
1) 채널 구조와 레지스터 슬라이스

마스터와 슬레이브 간은 쓰기 주소 채널, 읽기 주소 채널, 읽기 데이터 채널, 쓰기 데이터 채널, 쓰기 응답 채널이라는 다섯 가지 채널로 연결됩니다 (그림 6). AHB 사양과 같이 규정된 시간에 마스터에서 주소 또는 데이터 신호가 일방적으로 출력되는 것이 아니라, 읽기 동작에서 마스터와 슬레이브가 읽기 주소 채널과 읽기 데이터 채널을 통해 데이터를 전송한다. 쓰기는 쓰기 주소 채널, 쓰기 데이터 채널 및 쓰기 응답 채널을 통해 데이터를 전송한다.



[그림 6] AXI의 채널 구조

각 채널의 신호는 기본적으로 모두 같은 방향이므로 (Handshake Acknowledge 만 역방향), 채널의 경로 (path)에 Register 를 쉽게 삽입할 수 있습니다 (그림 7). 채널에 레지스터를 추가할 수 있다는 것은 타이밍이 중요한 설계에 큰 이점을 제공합니다. 버스가 고속 클럭에 동기화하여 작동할 경우 버스의 타이밍 설계는 매우 어려워집니다. 하지만, 신호선의 경로를 따라 레지스터를 삽입하는 것으로, 1 클럭 신호가 움직이는 범위가 줄어 배선 길이가 짧아집니다. 따라서 버스의 동작 주파수를 높일 수 있습니다.



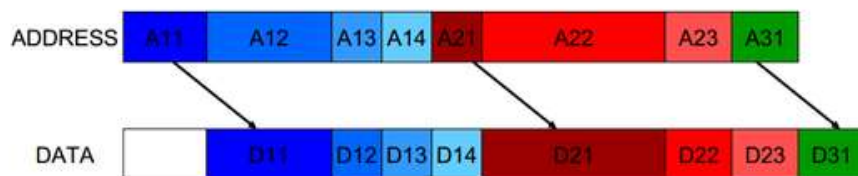
[그림 7] 채널 레지스터 삽입 (레지스터 슬라이스)

레지스터 슬라이스를 사용하여 전체 버스의 동작 주파수를 저하시키지 않고 버스 타이밍 설계를 할 수 있습니다. 그러나 레지스터를 삽입 한 위치에 데이터 전송을 하기 위해서는 1 클럭의 시간이 더 필요하게 됩니다. 고속 데이터 전송이 필요한 장치는 레지스터를 사용하지 않는 위치에 연결해야 합니다. (예; 이 그림의 Slave #1 에 연결해야 합니다.)

2) 버스트(Burst) 기반 트랜잭션

AXI 사양의 트랜잭션은 버스트(묶음) 전송을 전제로 하고 있습니다(물론 단일 데이터 전송도 할 수 있다). 고성능 시스템에서는 대부분의 경우 CPU 에 캐시 메모리가 탑재되어 있으며, 메인 메모리와의 데이터 전송은 캐시 메모리를 채우거나 메모리를 업데이트할 때 버스트 전송이 대부분을 차지하고 있습니다. 또한 지속적인 스트리밍 데이터는 일반적으로 일정량의 버스트 전송을 반복합니다. 따라서 버스트 전송을 사용할 경우 트랜잭션 시스템의 성능 향상을 기대할 수 있습니다.

AHB 에서는 버스트 전송시에 버스의 마스터가 데이터 주소를 하나씩 생성하는 방식을 사용했습니다. (그림 8). AXI 에서는 버스트로 전송되는 데이터의 주소는 첫 번째 주소만 보내고 후속 데이터의 주소 생성은 슬레이브가 책임을 집니다. 따라서 주소 채널의 전송량은 다른 채널에 비해 데이터의 양을 현격하게 줄일 수 있습니다.

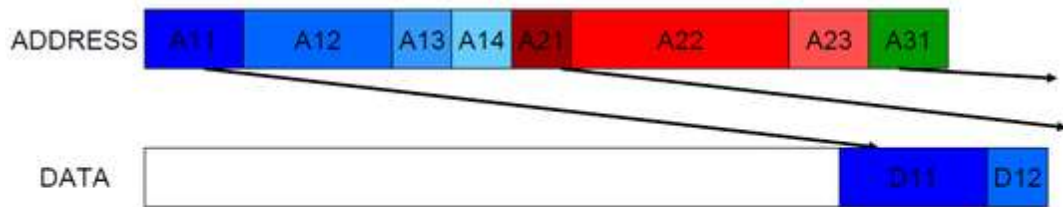


[그림 8] 버스트 전송 데이터의 주소는 선두만 지정

AHB 사양은 버스트 전송의 경우에도 각각의 주소를 마스터가 전송하였다.

3) 여러 주소 발행

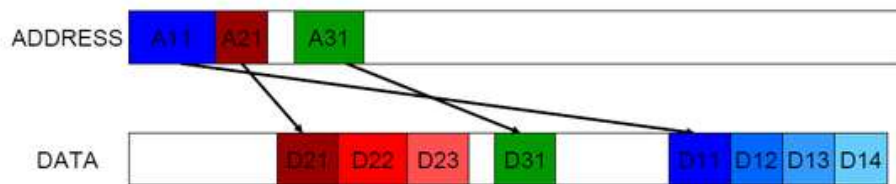
버스의 효율적인 이용이라는 관점에서 보면 저속 슬레이브 장치의 존재는 매우 신경이 쓰입니다. 기존의 버스는 하나의 트랜잭션이 종료한 후 다음을 수행하였습니다. 따라서 일단 저속 장치에 액세스 해 버리면, 그 트랜잭션이 끝날 때까지 전체 버스가 점령되어 버리는 문제점이 있었습니다.



- With AHB

- If one slave is very slow, all data is held up.

AXI 사양은 먼저 발행 한 주소의 트랜잭션이 종료되기 전에 다음 주소를 여러 게시할 수 있습니다 (여러 주소 발행). 발행된 주소에 해당하는 트랜잭션은 아웃 오브 오더 (out-of-order; 순서와 상관없이)에 완료할 수 있습니다(즉, 발행 한 주소의 순서와 관계없이 트랜잭션을 종료할 수 있다. 자세한 내용은 뒤에 설명하도록 하겠습니다.).



- Out of order completion allowed
- Fast slaves may return data ahead of slow slaves
- Complex slaves may return data out of order

4) 제어 정보 추가

고성능 시스템에서 요구하는 2 차 캐시를 연결하거나 프로세서에 추가된 보안 기능을 지원하기 위해 AXI 사양은 다음 신호를 정의하고 있습니다.

시스템 캐시를 연결하는 신호

- 보안 액세스를 나타내는 신호

5) 단독 액세스 추가

멀티 프로세서 구성의 경우, 프로세서 간의 동기화 동작을 실현하기 위해 세마포어 등이 이용됩니다. 세마포어를 실현하기 위해서는 세마포어 처리 동안 다른 프로세서 처리가 끼어들지 않도록 배타적 연산을 실시해야 합니다. AHB 는 이를 특정 버스 마스터가 버스를 점유하는 버스 잠금으로 구현하였고, 그 동안 다른 버스 마스터는 버스를 사용할 수 없도록 하였습니다. AXI 는 해당 주소에 단독으로 액세스하는 방법을 취하고 버스가 점령되지 않도록 되어 있습니다 (자세한 내용은 뒤에 설명하도록 하겠습니다.).

6) 바이트 레인 스트로브 신호

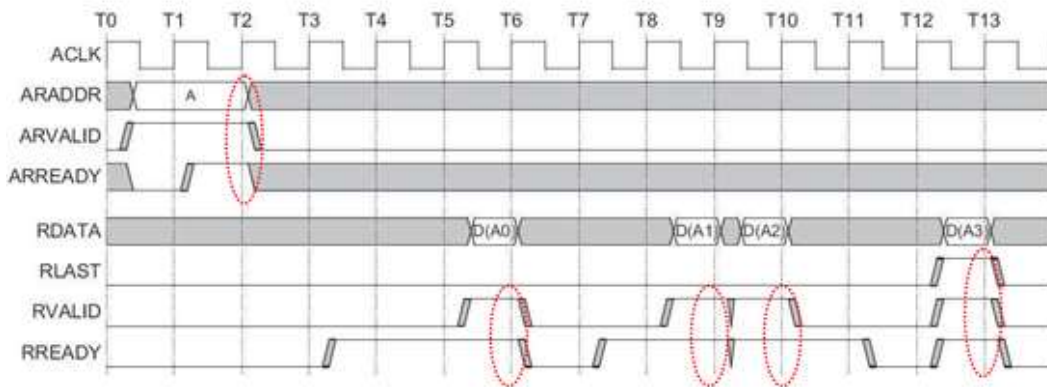
데이터 버스 폭보다 폭이 좁은 데이터를 전송하는 경우 어떤 바이트 레인에 유효한 데이터가 있는지를 나타내는 신호 (바이트 레인 스트로브 신호)가 추가되었습니다.

7) 저소비 전력화

소비 전력을 줄이기 위해 AXI 사양은 주변 회로의 클럭을 정지하기 위한 인터페이스 신호 및 프로토콜 사양이 정의되었습니다.

● 기본 데이터 전송 프로토콜

이번에는 AXI 사양의 기본 데이터 전송 프로토콜을 사용하여 읽기와 쓰기에 대한 4-Word 버스트 전송 트랜잭션을 예로 설명하도록 하겠습니다.



[그림 9(a)] Basic Read Burst Transaction

1) 읽기 트랜잭션 (그림 9 (a))

1. Master 의 주소채널은 전송 시작주소와 전송에 대한 제어 정보 (버스트 길이 데이터 폭 전송 유형 등)를 신고, ARVALID 신호를 Assert 합니다(그림 9(a) 시간 T0). 슬레이브는 주소 채널의 정보를 받을 수 있으므로, ARREADY 을 Assert 하고 응답합니다 (그림 9(a) 시간 T1). 주소 채널의 정보는 ARVALID 과 ARREADY 가 Assert 되는 시점에 전송됩니다. (그림 9(a)의 시간 T2)

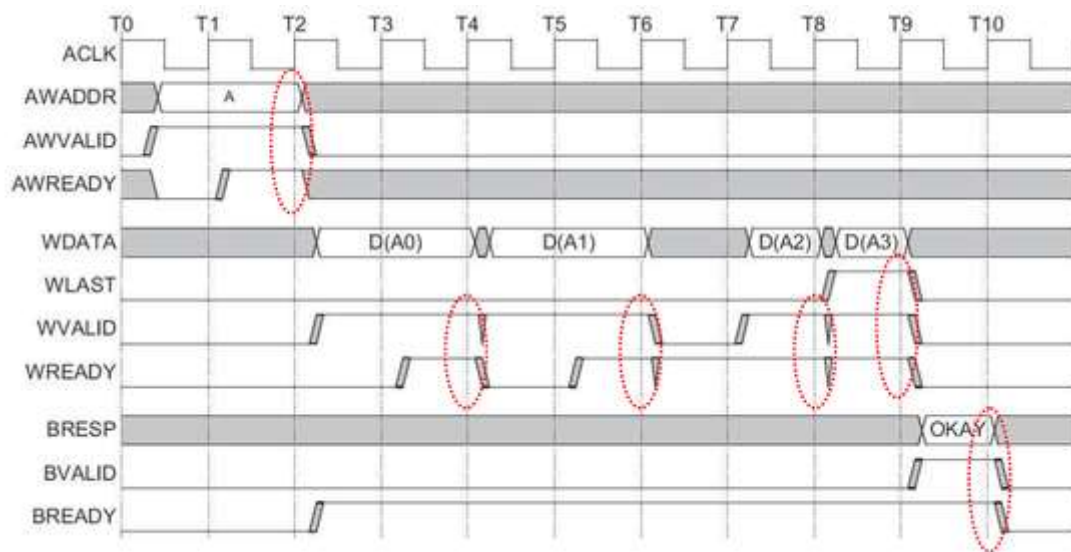
2. 시간 T5 에서 슬레이브의 읽기 데이터가 준비되어 슬레이브는 읽기 채널 RDATA 에 첫 번째 읽을 데이터를 실어 RVALID 신호를 Assert 합니다. 마스터는 데이터를 받을 준비가 되어 있었기 때문에 RREADY 이 Assert 되고, RVALID 과 RREADY 가 함께 Assert 되는 시간 T6 에 데이터가 전송됩니다.

3. 시간 T9 에서 읽기 데이터가 준비되어있는 슬레이브는 읽기 채널 RDATA 두 번째 읽을 데이터를 실어 RVALID 신호를 Assert 합니다. 마스터는 데이터를 받을 준비가 있었기 때문에 RREADY 이 Assert 되고, RVALID 과 RREADY 가 함께 Assert 되는 시간 T10 에서 데이터가 전송됩니다.

4. 같은 T10 에서 슬레이브는 다음 데이터가 준비되어 있기 때문에 읽기 채널 RDATA 세 번째 읽을 데이터를 실어 RVALID 신호를 Assert 합니다. 마스터는 데이터를 받을 준비가 있었기

때문에 RREADY 이 Assert 되고, RVALID 과 RREADY 가 함께 Assert 되는 시간 T11 에서 데이터가 전송됩니다.

5. 시간 T12 에서 읽기 데이터가 준비되어 있는 슬레이브는 읽기 채널 RDATA 에 4 번째 읽을 데이터를 실어 RVALID 신호를 Assert 합니다. 전송 마지막 데이터이므로 RLAST가 Assert 됩니다. 마스터는 데이터를 받을 준비가 있었기 때문에 RREADY 이 Assert 되고, RVALID 과 RREADY 가 함께 Assert 되는 시간 T13 에서 데이터가 전송됩니다.



[그림 9(b)] Basic Write Burst Transaction

2) 쓰기 트랜잭션 (그림 9 (b))

1. 마스터는 주소채널을 통해 전송 시작주소와 전송에 대한 제어 정보 (버스트 길이, 데이터 폭 전송, 유형 등)을 신고 AWVALID 신호를 Assert 합니다(그림 9(b) 시간 T0). 슬레이브가 주소채널의 정보를 받을 수 있으므로, AWREADY을 Assert 하고 응답합니다 (그림 9(b) 시간 T1). 주소채널의 정보는 AWVALID와 AWREADY가 Assert 되는 그림 9(b)의 시간 T2에 전송됩니다.

2. T2 이후 쓰기채널은 WDATA에 첫째 쓰기 데이터를 실어 WVALID 신호를 Assert 합니다. 슬레이브는 데이터를 받을 준비가 되어 있기 때문에, 시간 T3에서 WREADY을 Assert 합니다. WVALID과 WREADY가 함께 Assert 되는 시간 T4에 데이터가 전송됩니다.

3. 마스터는 다음 데이터를 준비할 수 있었으므로, T4에서 쓰기채널 WDATA에 두 번째 쓰기 데이터를 실어 WVALID 신호를 Assert 합니다. 슬레이브는 데이터를 받을 준비가 되어 있기

때문에 시간 T5 에서 WREADY 을 Assert 합니다. WVALID 과 WREADY 가 함께 Assert 되는 시간 T6 에서 데이터가 전송됩니다.

4. 마스터는 다음 데이터를 준비할 수 있었으므로, 쓰기 채널 WDATA 에 3 번째 기록 데이터를 실어 WVALID 신호를 Assert 합니다 (그림 9(b) 시간 T7). 슬레이브는 데이터를 받을 준비가 되어 있었기 때문에 WREADY 이 Assert 하고, WVALID 과 WREADY 가 함께 Assert 되는 시간 T8 에 데이터가 전송됩니다.

마스터는 동일한 T8 에 다음 데이터의 준비했기 때문에, 쓰기 채널 WDATA 에 4 번째 기록 데이터를 실어 WVALID 신호를 Assert 합니다. 전송 마지막 데이터이므로 WLAST 가 Assert 됩니다. WVALID 과 WREADY 가 함께 Assert 되는 시간 T9 에 데이터가 전송됩니다.

5. 같은 T9 에서 슬레이브가 쓰기 데이터를 받았는지 확인하고, 쓰기 응답 채널에 응답 상태를 신고 BVALID 을 Assert 합니다. 마스터는 응답 상태를 받을 준비가 되어 있었기 때문에 BREADY 가 Assert 되고, BVALID 와 BREADY 가 함께 Assert 되는 시간 T10 에서 BRESP 가 전송됩니다.

● 주소 채널에서 전송하는 전송 제어 정보

주소 채널로 전송되는 정보는 전송의 주소 외에도 다음과 같은 전송 제어 정보를 포함하고 있습니다.

1) 버스트 길이

주소 채널 ALEN [3:0]으로 전달되고 하나의 버스트 전송의 데이터 전송 횟수를 지정합니다. 버스트 길이는 버스트 유형에 따라 고정, 증가의 경우는 1 ~ 16, Wrap 의 경우는 2,4,8,16 중 하나입니다. 불확정 길이의 버스트는 정의되지 않습니다. 또한 지정된 전송 횟수에 도달하기 전에 버스트 전송을 중단할 수 없습니다. 버스트 전송 중에 주소가 4K 바이트의 주소 경계를 넘는 것은 불가능합니다.

ARLEN[3:0] AWLEN[3:0]	Number of data transfers
b0000	1
b0001	2
b0010	3
-	-
-	-
-	-
b1101	14
b1110	15
b1111	16

2) 버스트 크기

주소 채널 ASIZE [2:0]으로 전달되고 하나의 데이터 전송에서 전송되는 바이트 수를 지정합니다.
1,2,4,8,16,32,64,128 바이트에서 지정할 수 있습니다.

ARSIZE[2:0] AWSIZE[2:0]	Bytes in transfer
b000	1
b001	2
b010	4
b011	8
b100	16
b101	32
b110	64
b111	128

3) 버스트 유형

ARBURST[1:0] AWBURST[1:0]	Burst type	Description	Access
b00	FIXED	Fixed-address burst	FIFO-type
b01	INCR	Incrementing-address burst	Normal sequential memory
b10	WRAP	Incrementing-address burst that wraps to a lower address at the wrap boundary	Cache line
b11	Reserved	-	-

주소 채널 ABURST [1:0]으로 전달됩니다 버스트 전송 유형 (표 1)를 지정합니다.

버스트 유형	설명
Fixed burst	버스트 전송의 어떤 데이터 전송도 같은 주소에만 데이터를 전송하는 방식. FIFO 메모리 액세스와 같이 동일한 위치에 여러 데이터 전송에서 사용되는 경우 사용. □ Fixed burst ➤ 같은 주소로 데이터를 전송하는 방식 ➤ ex) 0x1000_0004, 4beat burst - transfer0 : 0x1000_0004 - transfer1 : 0x1000_0004 - transfer2 : 0x1000_0004 - transfer3 : 0x1000_0004 ➤ 같은 lane 사용
Incrementing burst	일반적인 버스트. 데이터 전송마다 주소가 일정하게 증가 해 나가는 방식

	□Incrementing burst ➤ 이전 주소에서 증가된 주소로 데이터를 전송하는 방식 ➤ ex) 0x1000_0004, 4beat burst - transfer0 : 0x1000_0004 - transfer1 : 0x1000_0008 - transfer2 : 0x1000_000C - transfer3 : 0x1000_0010 ➤ data size 만큼 증가 ➤ bus width보다 작은 데이터가 전송되는 경우 ✓ lane이 변경됨
Wrapping burst	증가 버스트처럼 주소는 일정하게 증가하지만, Wrapping 경계에 도달하면 주소 버스트는 더 증가하지 않고, 낮은 주소로 랩 어라운드 됨. CPU의 캐시 라인을 채울 때 사용. □Wrapping burst ➤ wrap 범위 안에서 주소가 증가하며 데이터를 전송하는 방식 ✓ Wrapping; (start_addr + data_size) mod 2ⁿ ➤ 적용 ✓ Circular Queue 구조체에 대한 접근 ✓ Cache의 critical word first operation. ➤ ex) 0x1000_0004, 4beat burst - transfer0 : 0x1000_0004 - transfer1 : 0x1000_0008 - transfer2 : 0x1000_000C - transfer3 : 0x1000_0000 ➤ bus width보다 작은 데이터가 전송되는 경우 ✓ lane이 변경됨

[표 1] 버스트 유형

4) 잠금 유형

주소 채널 ALOCK [1:0]으로 전달됩니다 Atomic 전송을 구현하기 위한 동작 모드 (표 2)을 지정합니다.

Atomic 전송은 어떤 객체에 대한 처리 과정이 다른 연산/처리에 의하여 방해받지 않고 이루어져야 하는 경우에 사용하며, 자원 사용에 대한 semaphore/MuTex 와 같은 데이터 구조의 처리에 사용된다.

잠금 유형	설명
일반적 접근	단독 처리를 하지 않은 일반 액세스임을 나타내는
단독 액세스	단독 액세스 (주소에 단독 처리를)임을 나타내는
록 액세스	록 액세스 (버스에 단독 처리를)임을 나타내는

[표 2] 잠금 유형

● 읽기 / 쓰기 응답 채널이 반환 상태 정보

읽기 트랜잭션의 경우, 슬레이브는 전송하는 데이터에 대해 마스터로 상태 정보를 보냅니다. 쓰기 트랜잭션의 경우, 슬레이브는 쓰기 데이터를 수신한 후, 상태 정보를 쓰기 응답 채널을 통해 하나 보냅니다. 두 경우 모두, 보내진 상태 정보는 표 3 중 하나입니다.

읽기 트랜잭션에서 16 word 버스트 전송의 경우, 16 개의 상태 정보를 보낼 수 있습니다. 상태 정보 중 하나가 오류가 나도 데이터 전송은 지정된 횟수만큼 실행하여 트랜잭션을 종료해야 합니다.

RRESP[1:0] BRESP[1:0]	Response	Meaning
b00	OKAY	Normal access okay indicates if a normal access has been successful. Can also indicate an exclusive access failure.
b01	EXOKAY	Exclusive access okay indicates that either the read or write portion of an exclusive access has been successful.
b10	SLVERR	Slave error is used when the access has reached the slave successfully, but the slave wishes to return an error condition to the originating master.
b11	DECERR	Decode error is generated typically by an interconnect component to indicate that there is no slave at the transaction address.

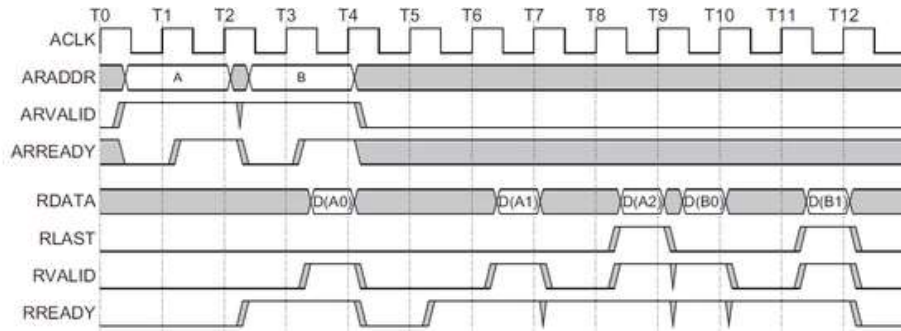
AXI(4) AMBA / ARM

2. AXI 버스 트랜잭션 예

그럼, 여기까지 설명한 내용에 대해 구체적인 예를 살펴보도록 하겠습니다.

● 트랜잭션 ID의 아웃 오브 오더 완료

AXI 사양은 트랜잭션 주소를 전달한 후 해당 트랜잭션에 대한 주소가 바로 전달되는 것은 아닙니다. 따라서 읽기채널 및 쓰기채널이 데이터를 전송하는 동안 주소채널은 다음주소 정보를 여러개 생성할 수 있습니다(그림 10). 주소 생성은 읽기채널의 데이터전송의 타이밍과는 관련 없습니다. 따라서 비동기 주소채널과 읽기/쓰기/쓰기응답채널 데이터 사이의 관계를 유지하기 위해 사용되는 것이 트랜잭션 ID 입니다.

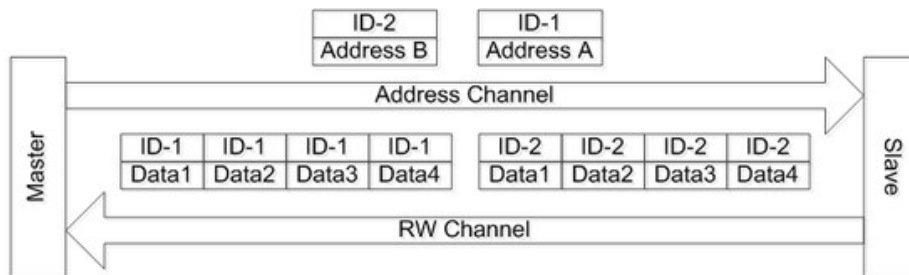


[그림 10] 읽기 버스트 전송의 복수 주소 발급

주소 채널이 트랜잭션 A 의 주소를 생성한 후, 트랜잭션 A 의 데이터 전송 과정을 기다릴 필요 없이 즉시 트랜잭션 B 의 주소를 생성할 수 있다.

1) 트랜잭션 ID 로 병렬 처리를 실현

트랜잭션 ID 는 트랜잭션의 시작 부분에서 주소정보와 함께 마스터에서 생성됩니다. 트랜잭션에서 전송되는 데이터에는 주소와 함께 생성된 트랜잭션 ID 가 반드시 생성되어 전송됩니다 (그림 11). 따라서 트랜잭션 ID 를 보면, 데이터는 어떤 전송에 관련되어 있는지를 확인할 수 있습니다.



[그림 11] ID 에 의한 데이터의 식별

트랜잭션에서 전송되는 데이터에는 주소와 함께 생성된 트랜잭션 ID 가 반드시 추가되어 전송된다. 이 ID 사용으로 비 순차 실행(Out of Order) 완료 및 데이터 인터리브 (다른 트랜잭션의 데이터 전송을 끼우는 것)이 가능하게 된다.

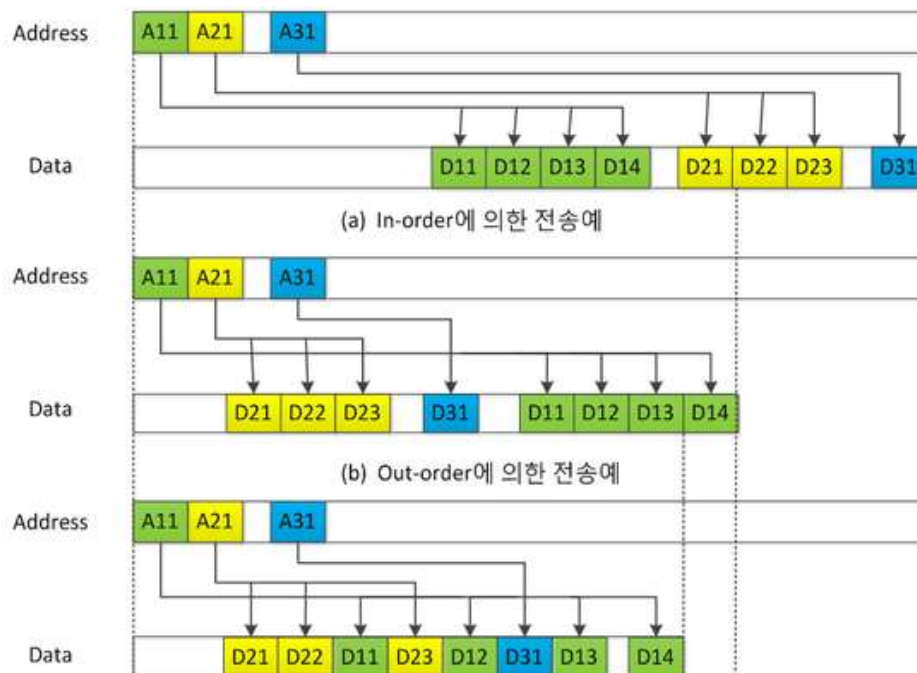
시스템에 여러 개의 마스터가 존재하는 경우, 각 마스터가 독자적으로 생성하는 ID 의 고유성이 문제가 됩니다. 이 경우 마스터와 슬레이브를 연결하는 인터콘넥터가 마스터마다 고유의 번호를 결정하고 그 번호를 트랜잭션 ID 의 상위에 연결하여 고유성을 유지합니다.

2) 느린 장치를 기다리지 않는 비 순차 실행

전송되는 데이터에 트랜잭션마다 고유 ID 를 추가함으로써 트랜잭션의 생성 순서에 맞춰 데이터가 도착하지 않아도 된다. (단, 같은 ID 를 가진 데이터 사이의 순서는 실행 순서와 동일해야 함.). 마스터는 순서가 아닌 트랜잭션 ID 에 의해 데이터를 식별하기 때문입니다. 이는 트랜잭션의 발행 순서에 관계없이 트랜잭션을 완료할 수 있습니다 (비순차실행: Out-of-Order).

비 순차 실행은 시스템의 성능을 향상시킵니다. 예를 들어, DRAM 과 같이 첫 번째 데이터가 출력 될 때까지 시간이 걸리는 메모리 전송 후, 빠른 메모리를 액세스하는 경우를 생각해 보겠습니다. 비 순차 실행이 불가능한 경우, 빠른 메모리에 액세스 했음에도 불구하고 느린 DRAM 의 동작을 기다려야 합니다 (그림 12 (a)).

비 순차 실행이 가능한 경우 나중에 액세스 한 빠른 메모리 전송을 먼저 완료한 이후 DRAM 의 전송을 완료할 수 있습니다. 그림 12 (b)는 자연이 큰 A11 트랜잭션의 완료를 기다리지 않고 A21, A31 의 트랜잭션을 먼저 완료합니다. 또한 여러 트랜잭션 데이터 전송을 그림 12 (c)와 같이 인터리브 (중간에 삽입하다는 의미) 수 있습니다.



[그림 12] 아웃 오브 오더와 데이터 인터리브 효과

비 순차 실행이 불가능한 경우, 먼저 액세스 한 메모리의 전송이 끝날 때까지 다른 메모리에 액세스 할 수 없습니다. 비 순차 실행이 가능한 경우 나중에 액세스 한 메모리의 전송을 먼저 완료할 수 있습니다. 또한 여러 트랜잭션 데이터 전송을 인터리브 할 수 있습니다.

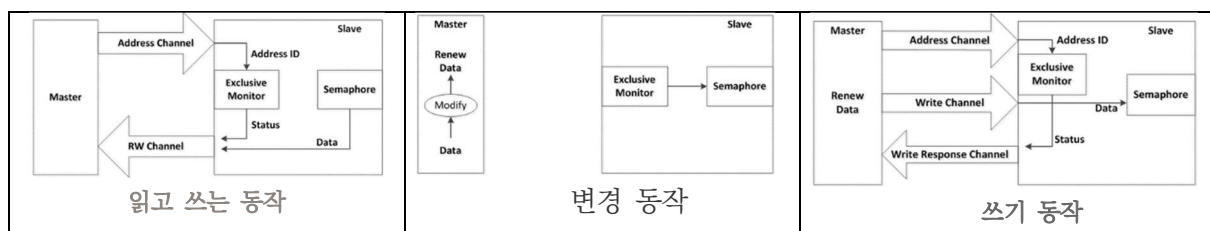
● 버스를 잠그는 단독 액세스

복잡한 시스템은 여러 개의 버스 마스터 간의 동기화 및 단독 처리를 실현하기 위한 수단으로, 세마포어 등이 필요합니다. 따라서 복잡한 시스템을 위한 버스는 세마포어를 구현하기 위해 배타적 연산을 지원하는 기능을 가질 필요가 있습니다.

세마포어의 실체는 여러 마스터간에 공유하는 특정 메모리 위치입니다. 동일한 메모리 내용을 마스터간에 참조/갱신하는 것으로 서로의 작동을 제어합니다. 세마포어가 제대로 작동하기 위해서는 어떤 마스터가 메모리를 읽기(read), 수정(modify), 쓰기(write) 동안 다른 마스터에 의해 방해되지 않도록(단독) 보장하는 메커니즘이 필요합니다.

AHB 는 이를 위해 잠금 신호를 정의하고 있습니다. 어느 마스터가 버스를 잠근 경우 다른 마스터가 버스를 획득할 수 없게 함으로써 독점적 처리를 할 수 있도록 했습니다. 이 방식의 문제점은 어느 마스터가 버스를 잠궈 버리면 다른 마스터는 버스를 획득하지 못하고, 잠금이 해제될 때까지 기다려야 하는 것입니다. AXI 에서는 버스를 잠그지 않고 단독으로 액세스 할 수 있습니다 (단, AHB 와 마찬가지로 잠금 액세스 실행할 수 있습니다.). 이 액세스는 슬레이브 쪽에 모니터가 필요하게 됩니다. 또한 버스트의 전송 길이는 1 로 제한됩니다.

다음은 단독 액세스의 일련의 흐름 (읽기, 변경, 쓰기)을 나타냅니다. (그림 13).



[그림 13] 단독 액세스 동작

단독 액세스의 일련의 과정을 보여준다.

1) 읽기 동작

마스터는 슬레이브를 읽을 때 단독 액세스를 사용합니다. 즉 ALOCK 신호를 단독으로 액세스하여 통지합니다. 슬레이브에서는 모니터가 단독 액세스를 감지하고 그 주소와 트랜잭션 ID 를 기억한 후, RESP 신호를 EXOKAY 상태로 응답합니다. 슬레이브가 단독으로 대응할 수 없는 경우는 OKAY 상태로 응답합니다. 이 경우 마스터는 단독으로 실패했다고 판단하고 Atomic 트랜잭션을 중단합니다.

2) 변경 작업

EXOKAY 상태를 수신한 마스터는 단독 처리를 계속하여, 읽어낸 데이터의 변경 작업을 수행합니다. 슬레이브에서 그 동안 기억 한 주소에 대해 다른 마스터에서 쓰거나 다른 주소에 대한 단독 액세스를 위한 읽기 동작이 발생하지 않는 한 감시를 계속합니다.

3) 쓰기 동작

마스터는 변경 데이터를 쓰는 것으로 단독 처리를 종료하려고 합니다. 슬레이브의 모니터가 해당 주소에 대해 모니터링을 계속하고 있으면, 변경 실행중인 배타성이 보장되어 있으므로, 슬레이브는 쓰기를 실시해, BRESP 신호에 EXOKAY를 돌려 단독 처리의 성공을 알립니다.

해당 주소에 대한 모니터링을 계속되지 않으면, 변경 동작중의 배타성이 보장되지 않기 때문에, 슬레이브는 쓰기를 하지 않고, OKAY 를 돌려 배타적 연산의 실패를 알립니다. 배타적연산이 실패하면 마스터 재시도되지만, 재시도한다고 해도 버스를 잠그는 기존 방식보다 시스템 전체의 성능 저하를 막을 수 있습니다.

참고 · 인용 * 문헌

- (1) ARM, AMBA Specification Rev 2.0,1999.
- (2) ARM, AMBA AXI Protocol Specification Rev r0p0, 2003.